DSA _ Assinment Report

Stu_name :Dhrubo Jouti Das Troyee

Stu_Id: 90028995

Table of Contents

Usage information:

- **Introduction:** **describing basic introduction (software). ----------2Page**
- **Dependencies: any libraries required to use (software) ----------6 page**
- **Terminologies and abbreviations used in the code ---------- 8page**
- **Future directions: suggested future improvements . -------- 9 page**

Class UML Diagrams:

- **Readme file, describing related information----- 10page**
- **Complete UML class diagrams ------16 page**
- **A complete association of classes/objects (i.e., class relationship) ---17page**
- **Complexity analysis of all operations performed by the software ----19page**
- **Traceability matrix of feature implementation and testing of code--- 20page**

Conclusion ---------- 20 page

Reference ---- 21 page

- **Usage information:**
- **Introduction: Basic Program Introduction_____**

The goal in this assignment is to create a smart system for traffic management by handling the critical issue of traffic congestion in urban areas. Using better understanding of abstract data types and algorithms, a powerful program that uses a variety of techniques to improve traffic flow, reduce congestion, and increase overall transportation efficiency.

The system is a flexible tool for dynamic traffic management since it is designed to make use of modern algorithms and data structures. Our application aims to offer flexibility for both urban planners and passengers by carefully analyzing traffic data, road networks, and current situations.

This introduction provides the basis for the assignment's following parts, which focus on the goals, approaches, and technology used to create this modern traffic optimization system.

- **Problem Description _____**

This program tackles the challenges of urban traffic management by providing features such as building a traffic network, improving traffic flow, dynamically managing traffic updates, and applying effective data handling techniques.

_____Task 1: Constructing a Traffic Network_____

Creating a graph representing the road network in an urban area using (traffic_network.txt) file which content data on data road connections, traffic flow, and distances between different locations in the map. Here Nodes in the graph denote different intersections or road segments, while edges will represent connections or roads between these locations. The output will be displaying the adjacency list representing the road network.

_____Task 2: Traffic Flow Optimization_____

Algorithms such as BFS and DFS Algorithm have been implemented to optimize traffic flow and identify efficient routes between different intersections. The system also uses BFS (Breadth First Search) to find the shortest path between locations and to explore all locations and path in the graph, it is using DFS (Depth First Search). From traffic data (traffic_data.txt) containing information on traffic volume, road capacity, and real-time congestion, we can ask the user for two locations as input and output will be Optimized traffic route between two locations entered.

____Task 3: Dynamic Traffic Management Implement_____

Here, we have implemented dynamic functionalities to update and modify the traffic network, adding new roads or intersections, modifying flow restrictions, searching for specific routes within urban areas.

However, the system also provides an interactive menu for users to dynamically update the traffic network by inputting new locations and paths. Users can add new locations, roads, or intersections, modify traffic flow and search for specific routes by using those methods.

_____Task 4: Efficient Data Handling_____

Here we are using hash tables for efficient management of traffic - related data (such as traffic volumes, congestion levels, and road conditions at various intersections or road segments.

Moreover, it is possible to quickly get data by adding edges to a hash table based on traffic_data.txt. The efficiency of hash table is compared against arrays and linked list double ended single linked list) for managing traffic data.

_____Task 5: Prioritized Traffic Control_____

Implemented a priority queue (heap) to prioritize traffic management strategies based on congestion levels, emergency situations, or roadwork. Using traffic volume as a priority to insert data in heap(max))

This system also assists in deploying traffic control measures effectively. Inserting the edges of the road network from text file (traffic_data.txt) to a heap helps to find out highest congestion edge, also sorting edged based on their congestion level.

Task 6: Advanced Traffic Optimization

Research and implement advanced algorithms such as Dijkstra's algorithm to further enhance traffic optimization beyond the simple DFS and BFS.

The system explores advanced algorithms to improve the accuracy and efficiency of route optimization. However, Within the urban network, Dijkstra's algorithm is regarded as an alternative method for determining the shortest paths between locations.

Task 7: Program Implementation and Testing

Here, we are trying to develop a program to manage traffic flow in urban areas, optimize routes, alleviate congestion, and improve transportation efficiency.

- **Again, this system includes an interactive menu for user interaction, error handling.**

❑ Dependencies:

Our traffic optimization system is written in Python and depends on several necessary modules to function properly. Here are those libraries required to use the program (software).

- **NumPy : which is a main python scientific programming package.**
- **Stack : It is already implemented by me.**
- **Queue : It is already implemented by me.**

- **Heap : For the following coding section we are using this part, implemented by me**

Using these dependencies our traffic optimization system is more reliable and effective, allowing the use of advanced algorithms and data structures. Moreover, to ensure smooth operation users are advised to install these libraries before starting the software. However, It is recommended to run the program on a Python 3.10 or updated environment.

❑ Terminologies and abbreviations

Certain terminology and abbreviations are used throughout the code to improve readability and maintain an easy-to-understand coding style. Gaining an understanding of these words is essential to understanding the program's logic and functionality.

- **ADT: Stands for Abstract Data Type, which is a high-level term describes a group of operation without specifying the implementation details.**
- **BFS: The algorithm known as BFS, or Breadth-First Search, is used to search or traverse tree or graph data structures.**
- **DFS: Depth-First Search, or DFS for short, is an algorithm that explores every branch of a tree or graph data structure to the maximum level possible.**
- **Hash Table: A hash table is a sort of data structure that links key to values by implementing the connected array abstract data type.**
- **Heap: A specific kind of tree-based data structure that satisfies with the heap requirement.**

- **Stack & Queue: Customize classes that represent the stack and queue data structures.**

- **Priority Queue "An ADT that functions similarly like a regular queue or stack, but with a priority assigned to each element.**

Future directions_____:

There are many suggested future implementations.

- **Machine Learning Integration: Implementation of machine learning for predict and adapt to traffic movements**

- **Real Time data Sources: Integrating real time traffic data more precisely from external sources such as traffic camaras, GPS, social media.**

- **User Interface enhancements: facilitate better user interaction and make it easier for traffic authorities and city planners to see and manage the traffic network.**

- **Scalability: The system also must be able to expand to meet the demands of cities that are expanding and changing road systems.**

- **Environment Impact Analysis: Integrate features for minimizing the environment impact of traffic, considering the factors also**

- **Multi Modal Transportation: Extend the system to consider and optimize multi modal transportation like walking, cycling and all**

❑ Class UML Diagrams_____

This project implements a traffic system using Python, with functionality for optimizing traffic routes and managing road networks.

❑ Readme file, describing related information

Files _____

testharness.py ----Interactive test harness for traffic network
traffic_info.py --- contains the methods for implementation of Graph class and others.
traffic_network.txt ----- file stores road network data
traffic_data.txt ----- text file contains traffic data
DSAHeapM.py --- customize heap implementation by me
DSAHeapEntryM.py ----- Customize heap implementation by me

In the "traffic_info .py "we have these methods to run the program.

- **Dijkra's_ algorithms ----- methods for finding the shortest path between two location**
- **BFS & DFS ------ implementation of algorithms**
- **traffic_flow_modify --- methods for modifying the traffic flow on a specific road segment.**
- **search_route --- methods for search for a route between two locations**
- **road_remove. --- methods for removing road from road network.**
- **add_new_location ---- methods add new locations on road network.**
- **optimize_traffic_route ---- methods for optimizing a traffic route based on traffic volume, road capacity, and congestion level.**
- **display_road_network --- methods for displaying the road network.**
- **add_edge ---- adding edge for start and ending location**
- **update_edge --- methods for update traffic data for exiting edge**
- **remove_intersection – methods for removing intersection from graph**
- **print_route -- methods to print a route with traffic info**
- **display_adj_lis --- methods to display adjacency list**
- **specific_route_search ----- methods for searching a specific route using a chosen algorithm**
- **efficiency_comparison ----- methods for comparing the efficiency between hash table, array, linked list**

Direction of How to use the program ----------

- ✓ **.To run the application**
  -----python testharness.py

- ✓ **Choose an option**
  ---- -Enter a number (1-16) to perform various operations

Menu:
1. Display Adjacency List
2. Find Route using BFS
3. Find Route using DFS
4. Add New Location
5. Add New Path
6. Update Traffic Data
7.Compare Data Structures Efficiency
8 Add new interaction
9. Modify traffic flow
10.Search Road
11.Road Remove
12 Road Intersection
13. Insert Edge with Traffic Data
14. Find Edge with Highest Congestion Level
15. Find Route using Dijkstra's Algorithm
16. Exit

- ✓ **Find Route using BFS**

-----Enter your choice: 2
Enter the start location: Downtown
Enter the end location: Central Park

- ✓ **Add New Location**

Enter your choice: 4
Enter the new location: Central Park
Location 'Central Park' added successfully.

- ✓ **Add New Path**

Enter your choice: 5
Enter the start location: Downtown
Enter the end location: Central Park
Enter the distance: 3

Path between 'Downtown' and 'Central Park' added successfully.

- ✓ **6. Update Traffic Data**

Enter your choice: 6
Enter the start location: Downtown
Enter the end location: Central Park
Enter the new traffic volume: 80
Enter the new road capacity: 100
Enter the new congestion level: Low
Traffic data for path 'Downtown' to 'Central Park' updated successfully.

- ✓ **7.Compare Data Structures Efficiency**

Enter your choice: 7
-----Efficiency Comparison----:
Hash Table Time: 0.0000322000 seconds
Array Time: 0.0000031000 seconds
Linked List Time: 0.0000026000 seconds

- ✓ **8 Add new interaction**

Enter your choice: 8
Add New Intersection:
Enter the name of the new intersection: Downtown
New intersection added: Downtown

- ✓ **9. Modify traffic flow**

Enter your choice: 9
Modify Traffic Flow Restrictions:
Enter the start location: Downtown

Enter the end location: Central Park
Enter the new traffic volume: 80
Enter the new road capacity: 100
Enter the new congestion level: Low
Traffic data updated for Downtown - Central Park

- ✓ **10.Search Road**

Enter your choice: 10
Search Specific Route:
Enter the start location: Downtown
Enter the end location: Central Park
Choose algorithm (BFS/DFS): bfs

BFS Path:
Edge: Downtown - Central Park | Traffic Volume: 80 | Road Capacity: 100 | Congestion Level: Low

- ✓ **11.Road Remove**

Enter your choice: 11
Remove Road:
Enter the start location: Downtown
Enter the end location: Central Park
Road removed: Downtown - Central Park

- ✓ **13. Insert Edge with Traffic Data**

Enter your choice: 13
Enter the road segment City Hall - Airport Road
Enter the traffic volume: 50
Edge inserted successfully.

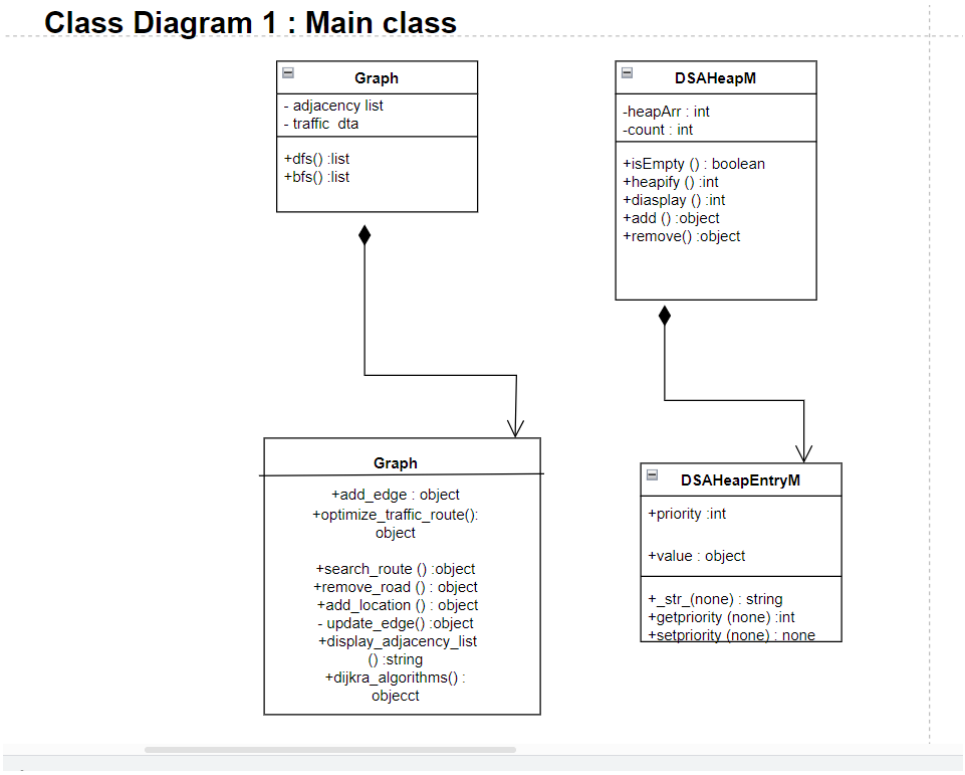
✓ **14. Find Edge with Highest Congestion Level**

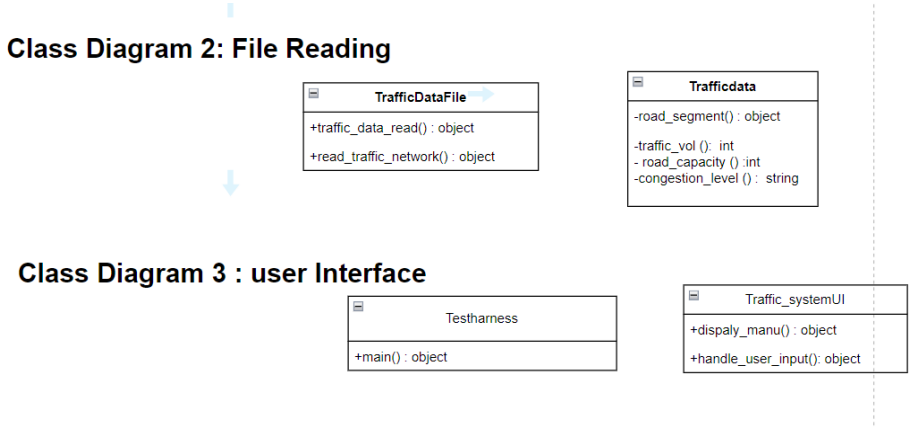
Enter your choice: 14
Highest Congestion Edge: ['North Avenue', 'Business District']

✓ **15. Find Route using Dijkstra's Algorithm**
Enter your choice: 15
Enter the start location: City Hall
Enter the end location: Airport Road
Shortest Path: ['City Hall', 'Airport Road']

✓ **16. Exit**
Enter your choice: 16
Exiting the program. Goodbye!

❑ **Complete UML class diagrams of the classes used for implementation.**





❑ A complete association of classes/objects (i.e., class relationship)

The classes are associated as follows:

- **The Graph class manages the road network and traffic data**
- **The DSAHeapM class implements the priority queue for traffic management.**

❑ Complexity analysis of all operations performed by the software

1.Graph construction (task1):
Reads and construct the graph from "network_graph_file" : O (V+E), where V is the number of Vertice and E is the number of edges.

2.BFS Algorithms (TAsk 2) :
BFS: O(V+E), where V is the number of Vertice and E is the number of edges.
DFS : O (V+E), where V is the number of Vertice and E is the number of edges.

Dynamics traffic Management: (task3) :
In this section depends on very specific task. For example, adding new locations or path will be O(1) or O(N).

Hash table efficiency Comparison(task 4):
Insert a data on hash table O(1) on average and it will be O(N) in worst case.

Priority Queue Operations(Task 5) :
Inserting an element into a max- heap will be O(logN), where N is the number of elements.
If someone wants to find the highest congestion edge that will be O(1)
Advanced traffic Optimization(task6) :

Dijkstra's algorithms : O((V + E) *log(V)), where log(V) is the complexity of heap operations. Here, V is the number of Vertice and E is the number of edges.

❑ Traceability matrix of feature implementation and testing of your code2.

Feature	Implementation	Testing

Task1 : constructing a traffic network	**Graph construction from text file "network_graph_file"**	**pass**
Task 2 : traffic flow optimization	**BFs Algorithms DFS Algorithms**	**Having issue with this.**
Task 4:Efficient Data handling	**Hash table for handling data (array, linked _list, hash table)**	**pass**
Task 5 : priority control Traffic	**Heap implementation**	**Check and test pass**
Task 6 : Advanced traffic optimization	**Dijkstra's algorithms**	**Check and pass**
Task 7 : program Implementation and testing	**Main program functionality**	**Check and pass successfully**

Conclusion:

Summarize the key findings of the project, highlight successes, and acknowledge any challenges faced. In conclusion, the development of the traffic optimization system has been a significant endeavor, aiming to address the challenges of urban traffic congestion through the implementation of various algorithms and data structures.

Reference :

geeksforgeeks. 2023. "What Is Dijkstra's Algorithm? | Introduction to Dijkstra's Shortest Path Algorithm." GeeksforGeeks. March 10, 2023.

https://www.geeksforgeeks.org/introduction-to-dijkstras-shortest-path-algorithm/.

Programiz. 2019. “Dijkstra’s Algorithm.” Programiz.com. 2019. https://www.programiz.com/dsa/dijkstra-algorithm.